

```

import logging
import random
import json

from telegram import Update, InlineKeyboardButton, InlineKeyboardMarkup
from telegram.ext import (
    Application, CommandHandler, CallbackQueryHandler,
    ContextTypes, MessageHandler, filters
)

TOKEN = "8506658443:AAHCv82zVYDBR0D0JW4BSbWNt75RwChfVd0"

logging.basicConfig(level=logging.INFO)

# =====
# 600 SO'Z - 30 UNIT
# =====

UNITS = {
1: [("afraid", "qo'rqqan"), ("agree", "rozi bo'lmoq"), ("angry", "g'azablangan"), ("arr
2: [("adventure", "sarguzasht"), ("approach", "yaqinlashmoq"), ("carefully", "ehtiyotk
3: [("alien", "o'zga sayyoralik"), ("among", "orasida"), ("chart", "jadval"), ("cloud",
4: [("appropriate", "mos, to'g'ri"), ("avoid", "qochmoq"), ("behave", "odob saqlamoq")
5: [("aware", "xabardor"), ("badly", "yomon"), ("belong", "tegishli bo'lmoq"), ("contin
6: [("advantage", "afzallik"), ("cause", "sabab"), ("choice", "tanlov"), ("community", "
7: [("allow", "ruxsat bermog"), ("announce", "e'lon qilmoq"), ("beside", "yonida"), ("c
8: [("accept", "qabul qilmoq"), ("arrange", "tartiblamog"), ("attend", "qatnashmoq"), (
9: [("against", "qarshi"), ("beach", "qirg'oq"), ("damage", "zarar"), ("discover", "kash
10: [("benefit", "foйда"), ("certain", "aniq"), ("chance", "imkoniyat"), ("effect", "ta'
11: [("anymore", "endi"), ("asleep", "uxlab yotgan"), ("berry", "rezavor"), ("collect",
12: [("alone", "yolg'iz"), ("apartment", "kvartira"), ("article", "maqola"), ("artist",
13: [("blood", "qon"), ("burn", "yonmoq"), ("cell", "hujayra"), ("contain", "o'z ichiga
14: [("coach", "murabbiy"), ("control", "boshqarmoq"), ("description", "tavsif"), ("dir
15: [("across", "orqali"), ("breathe", "nafas olmoq"), ("characteristic", "xususiyat")
16: [("academy", "akademiya"), ("ancient", "qadimiy"), ("board", "kengash"), ("century"
17: [("appreciate", "qadrlamoq"), ("available", "mavjud"), ("beat", "urmoq"), ("bright"
18: [("argue", "bahslashmoq"), ("communicate", "muloqot qilmoq"), ("crowd", "olomon"),
19: [("alive", "tirik"), ("bone", "suyak"), ("bother", "bezovta qilmoq"), ("captain", "k
20: [("achieve", "erishmoq"), ("advise", "maslahat bermog"), ("already", "allaqachon")
21: [("appear", "paydo bo'lmoq"), ("base", "asos"), ("brain", "miya"), ("career", "marta
22: [("actual", "haqiqiy"), ("amaze", "hayron qoldirmog"), ("charge", "haq olmoq"), ("c
23: [("advance", "ilgarilamoq"), ("athlete", "sportchi"), ("average", "o'rtacha"), ("be
24: [("block", "to'smog"), ("cheer", "quvvatlamog"), ("complex", "murakkab"), ("critic"
25: [("associate", "hamkor"), ("environment", "atrof-muhit"), ("factory", "zavod"), ("f
26: [("advice", "maslahat"), ("along", "bo'ylab"), ("attention", "diqqat"), ("attract",
27: [("actually", "aslida"), ("bite", "tishlash"), ("coast", "sohil"), ("deal", "bitim")

```

```

28: [ ("band", "guruh"), ("barely", "zo'rg'a"), ("boring", "zerikarli"), ("cancel", "beko
29: [ ("advertise", "reklama qilmoq"), ("assign", "tayinlamoq"), ("audience", "tomoshab
30: [ ("above", "ustida"), ("ahead", "oldinda"), ("amount", "miqdor"), ("belief", "ishonc
}

# =====
# DATABASE (xotira ichida)
# =====

users = {}      # user_id -> {name, total_score, tests, correct, wrong}
sessions = {}   # user_id -> {unit, questions, cur, score, correct, wrong}

def get_user(uid, name):
    if uid not in users:
        users[uid] = {"name": name, "total_score": 0, "tests": 0, "correct": 0, "
    return users[uid]

# =====
# KEYBOARDS
# =====

def main_menu_kb():
    return InlineKeyboardMarkup([
        [InlineKeyboardButton("📖 Unit tanlash", callback_data="units")],
        [InlineKeyboardButton("📊 Statistika", callback_data="stats"),
         InlineKeyboardButton("🏆 Reyting", callback_data="rating")],
        [InlineKeyboardButton("ℹ️ Yordam", callback_data="help")],
    ])

def units_kb(page=0):
    rows = []
    start = page * 10 + 1
    end = min(start + 9, 30)
    row = []
    for i in range(start, end + 1):
        row.append(InlineKeyboardButton(f"📖 {i}", callback_data=f"unit_{i}"))
        if len(row) == 5:
            rows.append(row)
            row = []
    if row:
        rows.append(row)
    nav = []
    if page > 0:
        nav.append(InlineKeyboardButton("⬅️ 1-10", callback_data="units_page_0"))
    if end < 30:
        nav.append(InlineKeyboardButton("11-30 ➡️", callback_data="units_page_1"))
    if nav:
        rows.append(nav)
    rows.append([InlineKeyboardButton("🏠 Bosh sahifa", callback_data="home")])

```

```

return InlineKeyboardMarkup(rows)

def options_kb(options, q_idx):
    rows = []
    letters = [" A", " B", " C", " D"]
    for i, opt in enumerate(options):
        rows.append([InlineKeyboardButton(f"{letters[i]}: {opt}", callback_data=f
rows.append([InlineKeyboardButton(" Testni tugatish", callback_data="end_te
return InlineKeyboardMarkup(rows)

def after_answer_kb():
    return InlineKeyboardMarkup([
        [InlineKeyboardButton(" Keyingisi", callback_data="next_q")]
    ])

def result_kb():
    return InlineKeyboardMarkup([
        [InlineKeyboardButton(" Qayta o'ynash", callback_data="retry")],
        [InlineKeyboardButton(" Boshqa unit", callback_data="units")],
        [InlineKeyboardButton(" Bosh sahifa", callback_data="home")],
    ])

# =====
# HANDLERS
# =====

async def start(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    user = update.effective_user
    get_user(user.id, user.first_name)
    text = (
        f" Salom, *{user.first_name}*\\!\\n\\n"
        " *4000 Essential English Words*\\n"
        "Bu bot sizga inglizcha so'zlarni test orqali o'rgatadi\\.\\n\\n"
        " *30 unit* – har birida *20 ta so'z*\\n"
        " To'g'ri javob – *+1 ball*\\n"
        " Reyting va statistika mavjud\\!\\n\\n"
        "Quyidagi menyudan tanlang:"
    )
    await update.message.reply_text(text, parse_mode="MarkdownV2", reply_markup=m

async def button(update: Update, ctx: ContextTypes.DEFAULT_TYPE):
    q = update.callback_query
    await q.answer()
    uid = q.from_user.id
    name = q.from_user.first_name
    data = q.data

    # HOME

```

```

if data == "home":
    text = (
        f" *Bosh sahifa*\n\n"
        f"Salom, *{name}*\!\! Nima qilmoqchisiz?"
    )
    await q.edit_message_text(text, parse_mode="MarkdownV2", reply_markup=mai

# UNITS LIST
elif data == "units" or data.startswith("units_page_"):
    page = 0
    if data.startswith("units_page_"):
        page = int(data.split("_")[-1])
    text = (
        f" *Unit tanlang*\n\n"
        "Har bir unitda 20 ta so'z bor\\.\n"
        "Unit raqamini bosing va test boshlang\!\!"
    )
    await q.edit_message_text(text, parse_mode="MarkdownV2", reply_markup=uni

# UNIT START
elif data.startswith("unit_"):
    unit_num = int(data.split("_")[1])
    await start_test(q, uid, name, unit_num)

# RETRY
elif data == "retry":
    if uid in sessions:
        unit_num = sessions[uid]["unit"]
        await start_test(q, uid, name, unit_num)
    else:
        await q.edit_message_text("❌ Avval unit tanlang.", reply_markup=unit

# ANSWER
elif data.startswith("ans_"):
    parts = data.split("_")
    q_idx = int(parts[1])
    chosen = int(parts[2])
    await handle_answer(q, uid, q_idx, chosen)

# NEXT QUESTION
elif data == "next_q":
    await send_question(q, uid)

# END TEST
elif data == "end_test":
    await show_result(q, uid)

```

```

# STATS
elif data == "stats":
    u = get_user(uid, name)
    acc = round(u["correct"] / max(u["correct"] + u["wrong"], 1) * 100)
    text = (
        f" Aniqlik: *{acc}%*"
    )
    kb = InlineKeyboardMarkup([[InlineKeyboardButton(" ", " "]
    lines = [" *Umumiy reyting \\\(Top 20\\\)*\n"]
    for i, (u_id, u_data) in enumerate(sorted_users):
        medal = medals[i] if i < 3 else f"{i+1}\\\."
        me = " ← Siz" if u_id == uid else ""
        lines.append(f"{medal} *{escape_md(u_data['name'])}* - {u_data['total
    text = "\n".join(lines)
    kb = InlineKeyboardMarkup([[InlineKeyboardButton(" Bosh sahifa", callba
await q.edit_message_text(text, parse_mode="MarkdownV2", reply_markup=kb)

# HELP
elif data == "help":
    text = (
        f" *Yordam*\n\n"
        f" *Qanday ishlaydi?*\n"
        f"1\\\ Unit tanlaysiz \\\(1\\\ - 30\\\)\n"
        f"2\\\ 20 ta savol beriladi\n"
        f"3\\\ 4 ta variantdan birini tanlaysiz\n"
        f"4\\\ To'g'ri javob  yoki xato  ko'rsatiladi\n"
        f"5\\\ Test oxirida natija chiqadi\n\n"
        f" *Ball tizimi:*\n"
        f"• To'g'ri javob: \\\ +1 ball\n"
        f"• Xato javob: 0 ball\n\n"
        f" Statistika va reyting mavjud\\!"
    )
    kb = InlineKeyboardMarkup([[InlineKeyboardButton(" Bosh sahifa", callba
await q.edit_message_text(text, parse_mode="MarkdownV2", reply_markup=kb)

```

```

# =====
# TEST LOGIC
# =====

async def start_test(q, uid, name, unit_num):
    get_user(uid, name)
    words = list(UNITS[unit_num])
    random.shuffle(words)
    sessions[uid] = {
        "unit": unit_num,
        "questions": words,
        "cur": 0,
        "score": 0,
        "correct": 0,
        "wrong": 0,
    }
    text = (
        f"🚀 *Unit {unit_num} – Test boshlandi\\!*\\n\\n"
        f"📖 20 ta savol\\n"
        f"✅ To'g'ri javob \\= \\+1 ball\\n\\n"
        f"Tayyor bo'lsangiz, birinchi savol\\!"
    )
    kb = InlineKeyboardMarkup([[InlineKeyboardButton("▶ Boshlash!", callback_data=
await q.edit_message_text(text, parse_mode="MarkdownV2", reply_markup=kb)

async def send_question(q, uid):
    if uid not in sessions:
        await q.edit_message_text("❌ Session topilmadi. /start bosing.", reply_markup=
        return

    s = sessions[uid]
    if s["cur"] >= len(s["questions"]):
        await show_result(q, uid)
        return

    word, correct_uz = s["questions"][s["cur"]]
    unit_num = s["unit"]
    all_words = UNITS[unit_num]

    # 4 ta variant yasash
    wrong_pool = [uz for en, uz in all_words if uz != correct_uz]
    wrongs = random.sample(wrong_pool, min(3, len(wrong_pool)))
    options = wrongs + [correct_uz]
    random.shuffle(options)

    correct_idx = options.index(correct_uz)
    s["current_correct"] = correct_idx
    s["current_options"] = options

```

```

cur = s["cur"]
total = len(s["questions"])
score = s["score"]
prog = "██" * (cur * 10 // total) + "░░" * (10 - cur * 10 // total)

text = (
    f"📖 *Unit {unit_num}* \\| {cur+1}/{total}\\n"
    f"{escape_md(prog)} *{score} ball*\n\n"
    f"📖 So'z:\n"
    f"_____\\n"
    f"*{escape_md(word.upper())}*\n"
    f"_____\\n\n"
    f"O'zbekcha tarjimasini toping:"
)

await q.edit_message_text(text, parse_mode="MarkdownV2", reply_markup=options)

async def handle_answer(q, uid, q_idx, chosen_idx):
    if uid not in sessions:
        return
    s = sessions[uid]
    if s["cur"] != q_idx:
        return

    correct_idx = s.get("current_correct", -1)
    options = s.get("current_options", [])
    word, correct_uz = s["questions"][s["cur"]]

    if chosen_idx == correct_idx:
        s["score"] += 1
        s["correct"] += 1
        result_line = f"✅ *To'g'ri\\!* \\+1 ball"
        emoji = random.choice(["🎉", "🌟", "💪", "🔥", "👏"])
    else:
        s["wrong"] += 1
        chosen_txt = escape_md(options[chosen_idx]) if chosen_idx < len(options)
        result_line = f"❌ *Xato\\!*\\nSiz: {chosen_txt}\\nTo'g'ri: {escape_md(correct_uz)}"
        emoji = "😓"

    s["cur"] += 1
    remaining = len(s["questions"]) - s["cur"]
    cur = s["cur"]
    total = len(s["questions"])
    prog = "██" * (cur * 10 // total) + "░░" * (10 - cur * 10 // total)

    text = (
        f"{emoji} *{escape_md(word.upper())}*\n\n"

```

```

        f"{result_line}\n\n"
        f"{escape_md(prog)} {cur}/{total} \\| *{s['score']} ball*"
    )

    if s["cur"] >= total:
        kb = InlineKeyboardMarkup([[InlineKeyboardButton("🇺🇦 Natijani ko'rish", c
    else:
        kb = after_answer_kb()

    await q.edit_message_text(text, parse_mode="MarkdownV2", reply_markup=kb)

async def show_result(q, uid):
    if uid not in sessions:
        await q.edit_message_text("🏠 Bosh sahifa", reply_markup=main_menu_kb())
        return

    s = sessions[uid]
    total = len(s["questions"])
    score = s["score"]
    correct = s["correct"]
    wrong = s["wrong"]
    unit_num = s["unit"]
    pct = round(score / max(total, 1) * 100)

    u = users.get(uid, {})
    u["total_score"] = u.get("total_score", 0) + score
    u["tests"] = u.get("tests", 0) + 1
    u["correct"] = u.get("correct", 0) + correct
    u["wrong"] = u.get("wrong", 0) + wrong

    if pct >= 90:
        emoji, grade = "🏆", "Mukammal\\!"
    elif pct >= 75:
        emoji, grade = "🌟", "Juda yaxshi\\!"
    elif pct >= 60:
        emoji, grade = "👍", "Yaxshi\\!"
    elif pct >= 40:
        emoji, grade = "💪", "Davom eting\\!"
    else:
        emoji, grade = "😓", "Ko'proq mashq qiling\\!"

    bar = "■" * (pct // 10) + "░" * (10 - pct // 10)

    text = (
        f"{emoji} *Unit {unit_num} – Natija*\n\n"
        f"*{grade}*\n\n"
        f"{escape_md(bar)} *{pct}%*\n\n"

```



```

        f"✅ To'g'ri: *{correct}/{total}*\\n"
        f"❌ Xato: *{wrong}/{total}*\\n"
        f"💰 Ball: *\\+{score}*\\n"
        f"📈 Jami ballingiz: *{u['total_score']}*"
    )

    del sessions[uid]
    await q.edit_message_text(text, parse_mode="MarkdownV2", reply_markup=result_

def escape_md(text):
    """Telegram MarkdownV2 uchun maxsus belgilarni ekranlash"""
    text = str(text)
    chars = ['_', '*', '[', ']', '(', ')', '~', '`', '>', '#', '+', '-', '=', '|']
    for c in chars:
        text = text.replace(c, f'\\{c}')
    return text

# =====
# MAIN
# =====

def main():
    app = Application.builder().token(TOKEN).build()
    app.add_handler(CommandHandler("start", start))
    app.add_handler(CallbackQueryHandler(button))
    print("✅ Bot ishga tushdi!")
    app.run_polling(drop_pending_updates=True)

if __name__ == "__main__":
    main()

```